Deliverable 44 — Framework Builder V2: five operator questions answered + two UX fixes shipped

Date: 2026-10-19 · **Runtime commit:** `3a85c5e` (deployed + verified on prod app + worker) · **Scope:** framework-builder UX only; scoring/evidence/analyze path untouched.

This deliverable answers the five questions raised while preparing to build a new framework in Builder V2, each grounded in the code, and documents the two in-scope fixes that were implemented, deployed, and verified. Question 5 (speed) is answered with the confirmed production numbers and a set of options that require explicit owner go-ahead before any runtime/cost change is made.

Q1 — "Enforce one proposal per message"

How it worked before. The intake prompt (`server/lib/framework-v2/intake-prompt.ts`) already carried an exhaustive "ONE QUESTION PER MESSAGE" rule (L30–45, restated L152/156/166, with anti-patterns). But the `/v2/chat` server handler (`server/routes/framework-builder-v2.ts`) did **not** enforce it deterministically — compliance was purely prompt-driven, so the LLM occasionally batched several questions/proposals into one turn. This was a *compliance gap, not a missing instruction*.

Fix shipped (#1). A deterministic server-side guard now runs on every intake turn. New pure module `server/lib/framework-v2/one-question-guard.ts` exports `detectMultiProposalTurn()` , which (after stripping fenced code blocks and option chips) flags a turn as a violation when it contains any of: ≥2 option-chip *groups*, ≥2 enumerated headers ("Question 1 / Step 2 / …"), ≥2 decision-bearing `?` -sentences, or a single sentence fusing ≥2 wh-questions. A compliant **one question + many `[[option:…]]` chips** passes untouched. On a detected violation the handler performs **exactly one** corrective re-generation constraining the model to emit only the single highest-priority next question; if the correction fails for any reason the original turn is kept (best-effort, never throws). **No content is dropped** — because intake is gap-driven, any held-back questions re-surface naturally on later turns.

Q2 — "Send to Claude after drafting only gives advisory recommendations — no way to incorporate them"

How it worked before (confirmed by design). "Send to Claude" at the review stage is `sendToClaudeBuilderReview()` → `POST /v2/builder-review` , rendered by `BuilderReviewPanel` . Those recommendations are advisory and are about the **shared framework BUILDER prompt** used by *every* framework — the panel even stated "nothing is applied automatically". They were never a path to edit the framework you are currently drafting, and the review-stage re-draft (`/v2/draft/refine`) only reran the LLM against the automatic *validation-violation* list — it did **not** accept free-text instructions. So after drafting there was genuinely no way to tell the builder "change X" in your own words until after a test drive. The confusion was real.

Fix shipped (#2). Two parts:

- **(2a) Clarified copy.** The `BuilderReviewPanel` banner now states unmistakably that these recommendations concern the **shared builder** (improving it for all future frameworks), are advisory, and are **not** applied to the current framework — and points to the new free-text refine for editing *this* framework.
- **(2b) Free-text refine at the review stage.** `/v2/draft/refine` now accepts an optional `instructions` string. When supplied, the re-draft is routed through the **normal full draft path** (`executeDraft` — complete validation + auto-repair, no bypass), with the instructions threaded through the drafting LLM calls as a generic verbatim passthrough that reaffirms the framework constraints. The `DraftReview` UI gained a textarea and a “Re-draft with these instructions” button. This is the proper place to act on anything the Claude builder-review surfaced, in your own words, before a test drive.

Q3 — “Can I re-edit the framework after drafting (e.g. after the test drive)?”

Yes — and this fix widens it. Capability by stage:

- **Review stage (post-draft, pre-test-drive):** previously only “Re-draft with corrections” (validation-driven) and “save as draft to edit measures manually later”. **Now also free-text refine (#2b)** — the gap at this stage is closed.
- **After a test drive (saved framework):** the strongest editing — the free-text `ImprovementChat` (`/v2/improvement/chat` → `/v2/improvement/apply` , which *does* apply), the `MeasureEditor` measure-edits, and `rescore` . Re-editing remains most powerful after a test drive, which is expected since the test drive supplies real-evidence signal to edit against.

Q4 — “I don’t see a step that asks me to validate specific examples during the builder phase”

It does happen — at intake, as robustness-gate item 9. During intake the builder proposes 4–6 candidate positive and 4–6 adversarial negative example snippets and asks you to accept-all or change-some (two option chips). It is likely this was accepted quickly or satisfied from a seeded intake, so it did not feel like a distinct “validate examples” step. There is **no dedicated “run these specific passages through the scorer” step before drafting** — example *snippets* are validated at intake (item 9), and examples are validated against **real disclosures at/after the test drive** (plus the post-run `truth-check` and `measure-drill` tools). So: snippet-level validation pre-draft, real-evidence validation at the test drive. No code change was required here — this is a “how it works” clarification.

Q5 — “How many companies process in parallel? It’s slow, and I need it much faster for thousands of companies”

Confirmed production numbers (read live from the Railway worker service env + `server/lib/capacity-config.ts` , `server/worker.ts`):

Setting	Value (worker service)
`WORKER_CONCURRENCY`	4
`CAPACITY_POLICY`	`dynamic`
`ADAPTIVE_CONCURRENCY_ENABLED`	`true`
`ADAPTIVE_CONCURRENCY_MIN`	2
`PG_POOL_MAX`	20
Worker replica count	unset → **1**

What this means. Effective parallelism is **up to 4 companies at a time on a single worker replica**, with the adaptive controller moving the live value in the **2–4** band depending on conditions. The `dynamic` capacity policy and adaptive controller are already armed, but the effective ceiling is pinned to 4 because `hardMax` tracks `WORKER_CONCURRENCY`. **The test drive uses the exact same worker and concurrency as production**, so test-drive slowness is this same ~4-wide bottleneck — not a separate “test mode” throttle.

Why it is capped here. The binding constraint is the hosted-Postgres connection ceiling (≈97 usable). `capacity-config.ts` keeps `concurrency × replicaCount × connectionsPerJob + overhead ≤ budget`, and the `dynamic` policy can only ever *lower* concurrency as replicas grow — never raise it above today’s static value. So the throughput levers are deliberate and connection-budget-aware, not accidental.

Options to go faster (NONE applied — each is a production runtime/cost change requiring explicit go-ahead):

Option	Direction	Effect	Caveat
A. Add worker replicas	Horizontal (owner-preferred)	Near-linear throughput; `dynamic` policy auto-keeps per-replica concurrency within the DB budget	Real limit is the Postgres connection ceiling at `PG_POOL_MAX=20`/replica — likely ~4 replicas before the pool must also be lowered; needs steady-state connection measurement first
B. Raise `WORKER_CONCURRENCY`	Vertical	Lifts the per-replica ceiling (also lifts `hardMax`)	Resilience fixes already shipped; still bounded by the same connection budget and single-replica CPU
C. Tune the adaptive band	Live	Raise `ADAPTIVE_CONCURRENCY_MIN`/ceiling so it spends more time at the top of the band	Only meaningful once A or B lifts the ceiling above 4
D. Lower `PG_POOL_MAX` + add replicas together	Horizontal	More replicas within the same connection ceiling	Requires verifying each worker's steady-state connection use first

Honest caveat. Throughput at scale has not been demonstrated beyond ~25 companies, so before scaling to thousands I recommend one measured step (confirm steady-state DB connections per worker at concurrency 4) so we pick replica count / pool size from evidence rather than the budget model's assumptions. I have **not** changed concurrency, replicas, pool size, or any policy — those await your explicit instruction.

Verification summary

- **Build/tests (at `3a85c5e`):** `tsc --noEmit` → 62 pre-existing errors, **0 new** in changed files. New `one-question-guard.test.ts` **12/12**. Full framework-v2 suite **368/368** (with `DATABASE_URL` set).
- **Deploy parity:** pushed to `origin/main`; Railway verified **prod app + worker both on `3a85c5e`, SUCCESS**. Local HEAD == origin/main == both prod targets. *Tested code == production code.*
- **Cell-flip (constraint):** builder-UX changes only — no scoring run, no evidence path touched ⇒ **Yes→No = 0, No→Yes = 0, total flips = 0** by construction.

- **Agnosticism (constraint):** #1 detector and #2 refine are generic (no topic/company/framework-specific logic; instructions are a verbatim passthrough).
- **Constraint 4:** no human-in-the-loop gate added to the analyze/worker path.